

Ön Gereksinimler

CEN207 Veri Yapıları — ilk güne ne getirmeli

Dr. Öğr. Üyesi Uğur CORUH · 2026-2027 Güz

Bu neden önemli — 1. haftadan itibaren

- İlk haftadan itibaren **kendi bilgisayarınızda program yazıyor, derliyor ve test ediyorsunuz**
- Vize projesi için önce **C**, final projesi için ardından **Java**
- Çalışmanızı baştan sona **Git ve GitHub** üzerinde yürütüyorsunuz
- Bu yüzden bu sunumdaki bilgi ve araçlarla gelmeniz gerekir

Resmî ön koşullar

- CEN107 Algoritmalar ve Programlama I
- CEN108 Algoritmalar ve Programlama II

CEN107'nin ilk haftalarında öğretilen geliştirme ortamı, Git, birim test ve şablon kullanımı bu derste **bilindiği varsayılarak** kullanılır.

İlk dersten önce: bu sunumun sonundaki kendinizi sınaama görevlerini kendi bilgisayarınızda çözün.

1. CEN107/CEN108'den gelenler

Geliştirme ortamı

- Derleyici (GCC/Clang/MSVC), IDE, Windows'ta WSL, CMake ile derleme
- Öğretildiği yer: **CEN107 Hafta 2 — Geliştirme ortamları**
- Burada kullanıldığı yer: 1. haftanın C atölyesi; vize (C) projesinin kurulumu

Git ve GitHub

- Depo oluşturma, `clone`, `commit`, dal (branch), `pull request`, `.gitignore`
- Öğretildiği yer: **CEN107 Hafta 3 — Git ile sürüm yönetimi**
- Burada kullanıldığı yer: proje rehberinin iş akışı (şablondan depo, plan, teslim) — her iki kontrol noktası için

Birim test ve kapsam araçları

- Test yazma, çalıştırma, kapsam (coverage) raporu okuma
- Öğretildiği yer: **CEN107 Hafta 4 — Birim test ve kütüphaneler**
- Burada kullanıldığı yer: vize rubriği (GoogleTest, gcov/lcov) ve final rubriği (JUnit 5, JaCoCo)

Proje şablonlarının kullanımı

- Şablondan özel depo oluşturmak ("Use this template", fork değil), derlemek, testlerini ve dokümantasyonunu üretmek
- Öğretildiği yer: **CEN107 Hafta 2–4**
- Burada kullanıldığı yer: proje rehberindeki `cpp-cmake-ctest-template` (vize) ve `eclipse-java-maven-template` (final)

Temel algoritma analizi ve özyineleme

- İşlem sayısını saymak, basit bir özyinelemeli fonksiyon yazmak
- Öğretildiği yer: **CEN108, ilk haftalar**
- Burada kullanıldığı yer: 1. haftada Big-O hatırlatması; 3. haftadan itibaren özyinelemeli çözümler (Hanoi Kuleleri, DFS, ağaç dolaşımları, birleştirmeli/hızlı sıralama)

2. C ve Java programlama temelleri

C temelleri (vize projesi)

- İşaretçiler ve işaretçi aritmetiği (`int *p;` , `p + 1` , `p` ile `*p` farkı)
- Diziler: tanımlama, indeksleme, fonksiyona geçirme
- `struct` tanımları; alanları bir arada tutmak (örn. bağlı liste düğümü)
- Dinamik bellek: `malloc` / `free` , her `malloc` 'un neden bir `free` gerektirdiği
- Özyineleme: kendini çağıran bir fonksiyon, taban durumuyla birlikte
- Dosya G/Ç, özellikle **binary dosyalar** (`"rb"` / `"wb"` ile `fopen` , `fread` , `fwrite`)

Java temelleri (final projesi)

- Sınıflar ve nesneler: alanlar, yapıcılar (constructor), metotlar, `this`
- Jenerik temelleri: `List<T>` kullanmak, basit bir jenerik sınıf/metot tanımlamak
- İstisnalar: `try` / `catch` / `finally` , denetimli/denetimsiz istisna farkı

3. Derste kısaca hatırlatılanlar

Ayrıntısıyla hatırlamasanız sorun değil — 1. hafta, kullanılmadan önce kısa bir hatırlatma yapar.

Ön bilgi	Hangi haftada gerekiyor?
Big-O gösterimi: işlem sayısını saymak, büyüme hızlarını karşılaştırmak	1. hafta, ardından veri yapılarını karşılaştırmak için her hafta
Bellek yerleşimi: yığın (stack) ve heap	1. hafta, ardından işaretçi/dinamik bellek kullanılan her yerde

4. Bilgisayarınızda kurulu olması gerekenler

Gerekli araç zinciri

- C derleyicisi: Windows'ta **Visual Studio 2022** (C++ ile masaüstü geliştirme), Linux/WSL/macOS'ta **GCC/Clang**
- **CMake** — vize şablonu bununla derlenip test edilir
- C kapsam aracı: Linux/WSL'de **gcov/lcov**, Windows'ta MSVC ile **OpenCppCoverage**
- **Doxygen** — proje dokümantasyonunu üretir
- **JDK 21** ve **Maven** — Maven, JUnit 5 ve JaCoCo'yu otomatik getirir
- **Git** ve bir **GitHub** hesabı

Kurulum adımları ve proje şablonlarının kendisi: **proje rehberi**.

5. İlk dersten önce kendinizi sınavın

Görev 1 — Derleyip tahmin edin

Bu program ne yazdırır?

```
int x = 5;  
int *p = &x;  
*p = *p + 1;  
printf("%d %d\n", x, *p);
```

Görev 1 — Cevap

6 6

`p`, `x`'in adresini tutar; bu yüzden `*p` ile `x` aynı bellek konumunu gösterir. Birini değiştirmek diğerini de değiştirir.

Görev 2 — İşaretçi aritmetiği

`int arr[4] = {10, 20, 30, 40}; int *p = arr;` verildiğinde, `*(p + 2)` ve `p[2]` ifadelerinin değeri nedir?

Görev 2 — Cevap

İkisi de `30` . `p[2]` , tanım gereği `*(p + 2)` anlamına gelir — dizi indeksleme, farklı yazılmış işaretçi aritmetiğinden başka bir şey değildir.

Görev 3 — Hatayı bulun

```
int *make_array(int n) {  
    int arr[n];  
    for (int i = 0; i < n; i++) arr[i] = i * i;  
    return arr;  
}
```

Ne yanlış, nasıl düzeltirsiniz?

Görev 3 — Cevap

`arr`, **yığın (stack)** üzerinde yaşayan yerel bir dizidir; fonksiyon döndüğünde yok olur, dolayısıyla dönen işaretçi **sahipsiz (dangling)** kalır. Çözüm, **heap** üzerinde ayırmaktır —

```
int *arr = malloc(n * sizeof(int));
```

— ve çağıranın, ihtiyaç kalmadığında `free` ile serbest bırakmasıdır.

Görev 4 — Yapı (struct) ve özyineleme

`struct Node { int value; struct Node *next; };` tanımlayın; düğüm sayısını döndüren özyinelemeli `int length(struct Node *head)` yazın.

`length(NULL)` ne döndürür ve özyineleme neden orada durur?

Görev 4 — Cevap

```
int length(struct Node *head) {  
    if (head == NULL) return 0;  
    return 1 + length(head->next);  
}
```

`length(NULL)` `0` döndürür — **taban durum (base case)**; bu olmadan özyineleme hiç durmaz.

Görev 5 — Binary dosya G/Ç

```
int values[3] = {1, 2, 3};  
FILE *f = fopen("data.bin", "wb");  
fwrite(values, sizeof(int), 3, f);  
fclose(f);
```

`data.bin` kaç bayt içerir, neden?

Görev 5 — Cevap

`3 * sizeof(int)` bayt — günümüz masaüstü platformlarının neredeyse tamamında **12 bayt**. `fwrite`, dizinin ham baytlarını doğrudan dosyaya kopyalar; `fprintf` 'in aksine metin biçimlendirmesi yoktur.

Görev 6 — Java jenerikleri

```
List<Integer> numbers = new ArrayList<>();  
numbers.add(10);  
numbers.add(20);  
int sum = 0;  
for (int n : numbers) { sum += n; }  
System.out.println(sum);
```

Bu program ne yazdırır?

Görev 6 — Cevap

30

`List<Integer>`, yalnızca `Integer` elemanlarına izin veren jenerik bir koleksiyondur; for-each döngüsü her elemanı okuyup `sum` 'a ekler.

Görev 7 — Java istisnaları

```
try {  
    System.out.println("A");  
    throw new RuntimeException("boom");  
} catch (RuntimeException e) {  
    System.out.println("B");  
} finally {  
    System.out.println("C");  
}
```

Üç harf hangi sırayla yazdırılır?

Görev 7 — Cevap

A
B
C

İstisna, eşleşen `catch` tarafından hemen yakalanır (`B`); `finally` ise istisna fırlatılsın ya da fırlatılmasın her zaman çalışır (`C`).

Görev 8 — İki araç zincirini de derleyip test edin

C araç zinciri: `gcc --version` · `cmake --version` ·

`cpp-cmake-ctest-template` 'i klonlayın ·

`cmake -S . -B build && cmake --build build` · `ctest --test-dir build`

Java araç zinciri: `java -version` · `mvn -version` ·

`eclipse-java-maven-template` 'i klonlayın · `mvn test`

Görev 8 — Başarı nasıl görünür

- `ctest` : 100% tests passed
- Maven: **BUILD SUCCESS**, Failures: 0
- Her iki araç zincirinde de yol boyunca hiçbir derleyici hatası yok

Biri başarısız olursa: "Kurulu olması gerekenler"e dönün, eksik aracı kurun, ilk dersten önce yeniden deneyin.

Derse hazırsınız

Sekiz görevin de mantıklı geldiyse ve iki araç zinciri de hatasız derlenip test edildiyse hazırsınız.

Sırada: haftalık programı gösteren **izlence** ve her kontrol noktasının tam rubriğini içeren **proje rehberi**.